Algoritmi paraleli și distribuiți

Arhitecturi de calcul paralel/distribuit

+ Java paralel

Ce este ”arhitectura unui sistem de calcul paralel”

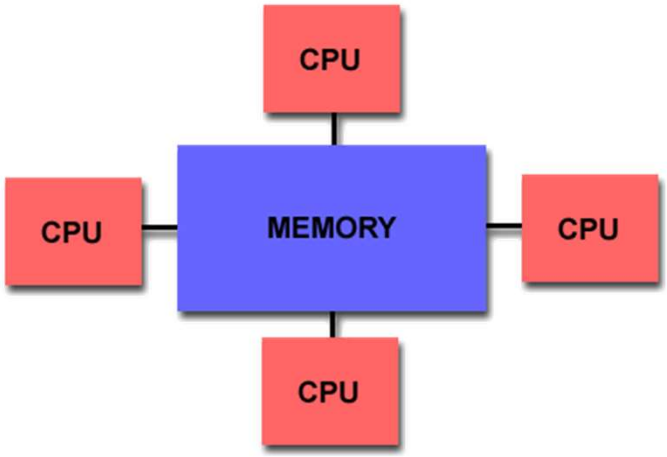
- Arhitectura unui sistem de calcul paralel trebuie să aibă în vedere
  - Alocarea resurselor
    - Cât de mare este colecția de date administrată?
    - Cât de puternice sunt elementele/componentele sistemului?
    - Câtă memorie au elementele sistemului?
  - Accesul la date, comunicare și sincronizare
    - Cum comunică elementele sistemului?
    - Cum sunt transmise datele între procesoare?
    - Care sunt primitivele de cooperare?
  - Performanță și scalabilitate
    - Cum se traduce totul în termeni de performanță?
    - Cum scalează sistemul?

Arhitecturi

- Cu memorie partajată
- Cu memorie distribuită
- Hibride: cu memorie partajată și distribuită

Sisteme paralele cu memorie partajată

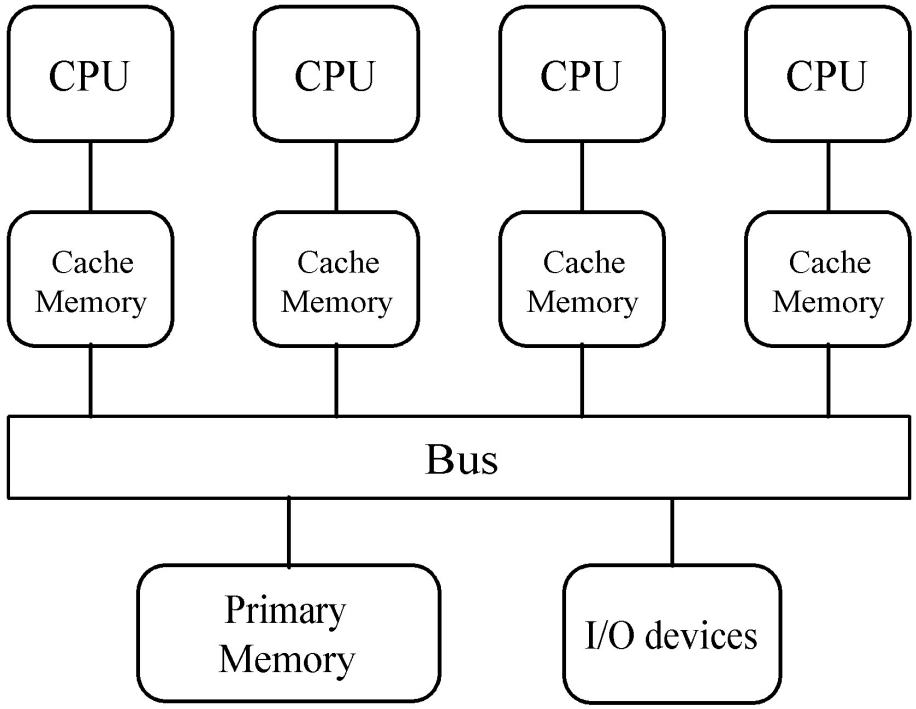
- Toate procesoarele accesează toată memoria. Aceasta este disponibilă într-un spațiu de adresare global.
- Procesoarele funcționează/execută instrucțiuni independent unele de altele dar partajează aceleași resurse de memorie.



- Modificările efectuate intr-o locație de memorie de către un procesor sunt vizibile de către toate celelalte procesoare.
- În funcție de timpul de acces la memorie sistemele cu memorie partajată se împart în două categorii: **UMA** și **NUMA**.

UMA [Uniform Memory Access]

- Sunt reprezentate de masinile **SMP** (en. ***S**ymetric **M**ultiprocessor **M**achines*)
- Procesoarele sunt identice și au același timp de acces la memorie și drepturi de acces egale.
- Dacă un procesor face un update în memorie atunci toate celelalte procesoare sunt informate despre update.
  - CC-UMA = Cache coherent UMA
- Coerența cache-ului este realizată la nivel HW.
- NU au mai mult de cateva zeci de procesoare.



NUMA [Non-Uniform Memory Access]

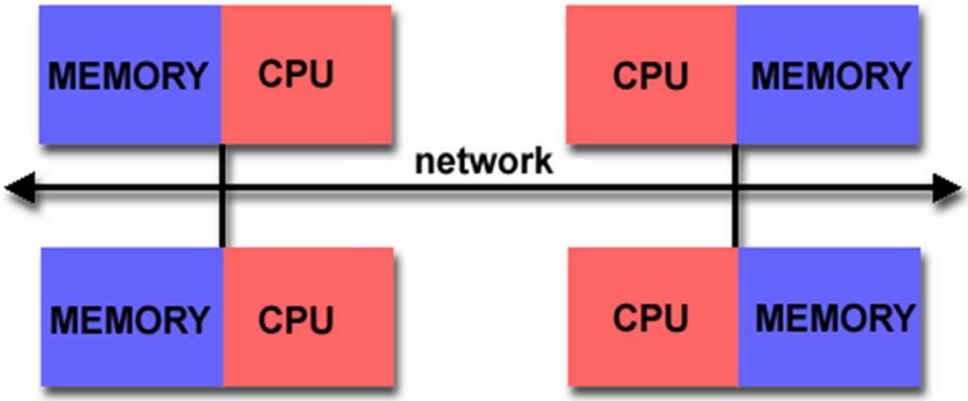
- Se obține de obiei prin conectarea fizică a două sau mai multe SMP-uri (en. ***S**ymetric **M**ultiprocessor **M**achines*).
- Un SMP poate accesa direct memoria altui SMP.
- Nu toate procesoarele au timp egal de acces la toate memoriile.
- Timpul de acces prin legatura la alt SMP este mai mare.
- Dacă se menține coerența cache-ului atunci sistemul este de tip CC-NUMA [**C**ache **C**oherent NUMA]

NUMA: Partajarea memoriei

- Avantaje
  - Spațiul de memorie oferă o perspectivă ușor de folosit pentru programator.
  - Partajarea datelor între task-uri este rapidă și uniformă datorită apropierii memoriei de procesoare.
- Dezavantaje
  - Lipsa scalabilității între memorie și procesoare.
    - Adăugarea de procesoare poate crește traficul pe o cale partajată intre memorie si un procesor. Pentru sistemele care mențin coerența chache-ului vor apărea creșteri semnificative ale traficului asociat activității de administrare a cache-ului/memoriei.
  - Este responsabilitatea programatorului pentru asigurarea sincronizarii care să asigure acces ”corect” la memoria globală. [adresarea *nedeterminismului*]
  - Cost: Proiectarea și realizarea sistemelor este din ce în ce mai dificilă pe măsură ce crește numărul de procesoare.

Sisteme paralele cu memorie distribuită

- Memoriile distribuite necesită o rețea de comunicare care să conecteze memoriile procesoarelor.
- Procesoarele au memorie proprie, locală. Adresele de memorie ale unui procesor nu corespund cu cele ale altui procesor, așadar nu există conceptul de spațiu de memorie global accesibil tutror procesoarelor

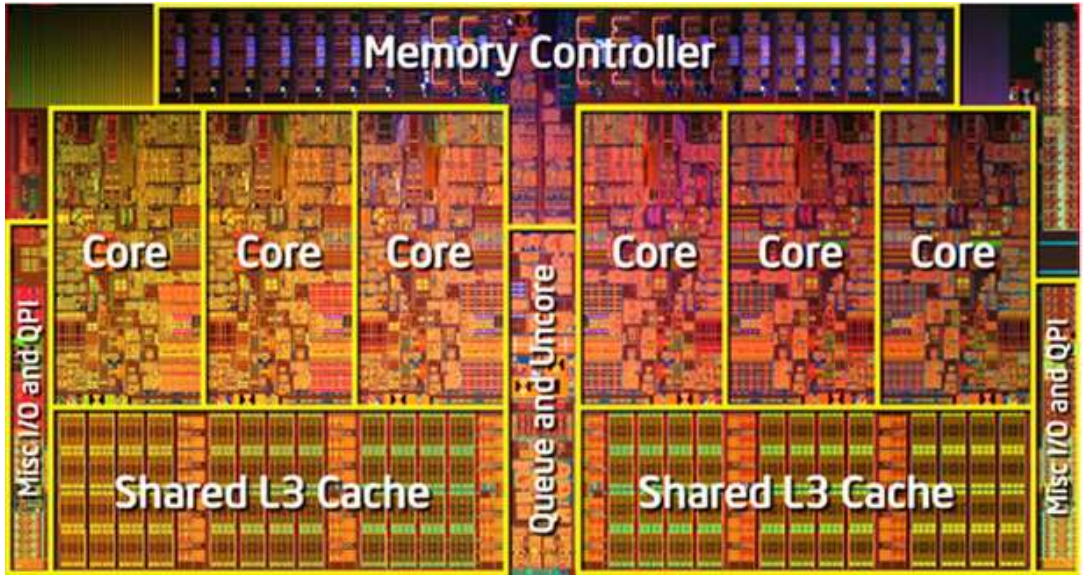


- Fiecare procesor operează/execută independent. Astfel, modificările efectuate în memoria sa locală nu au nici un efect asupra celorlalte procesoare. Connceptul de coerență a cache-ului nu există.
- Atunci când un procesor dorește să acceseze date ale altui procesor este de obicei responsabilitatea programatorului să specifice explicit cum și când sunt datele tranmsmise. Sincronizarea între task-uri este responsabilitatea programatorului.

SMP: Sisteme cu memorie partajată

- Procesorul Intel® Core™ i7-970

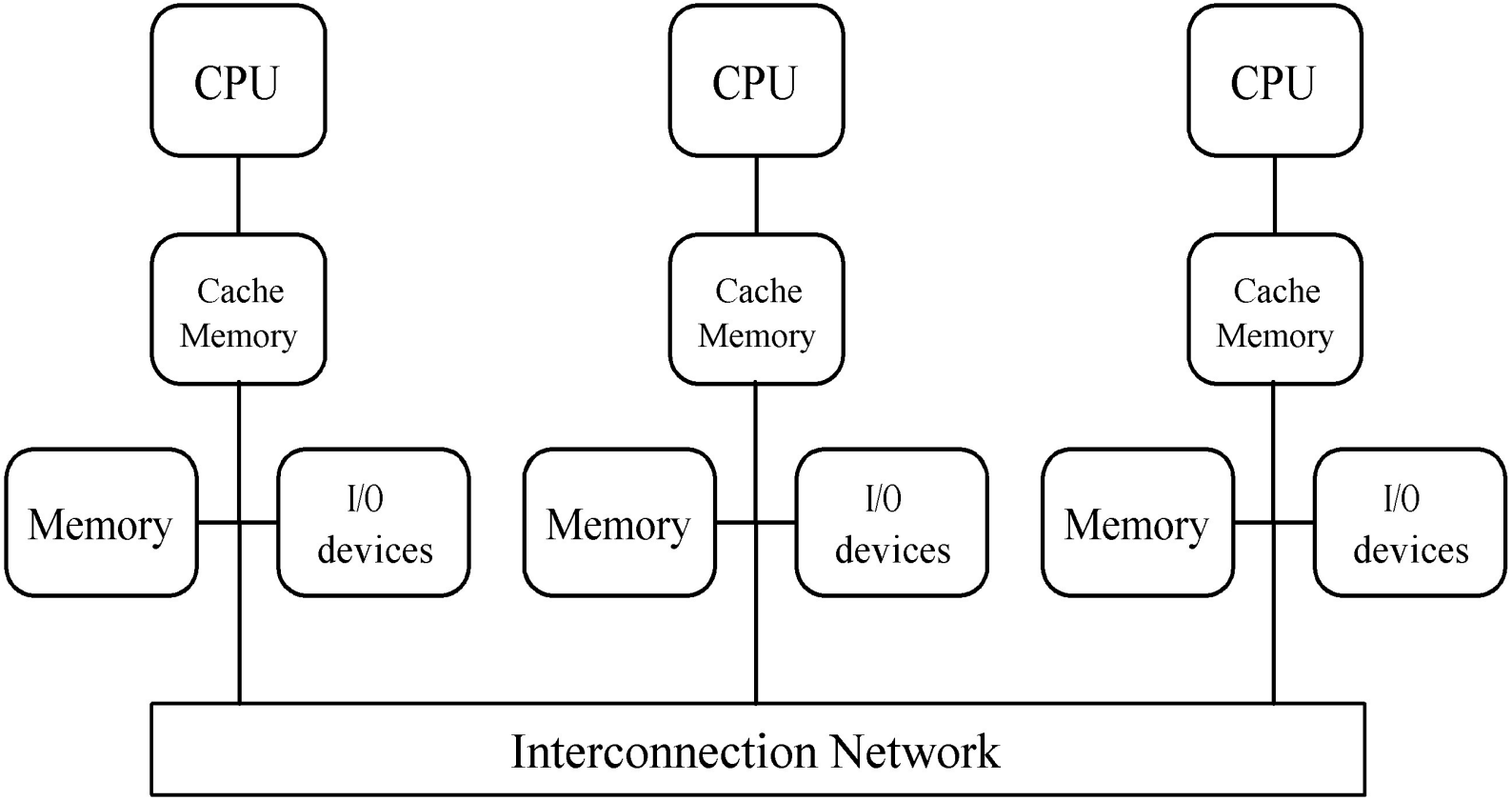
este adeseori denumit SMP!



https://bit-tech.net/reviews/tech/cpus/intel-core-i7-970-review/1/

- 6 core-uri, 2-way multithread, 4-wide SIMD (pe 3 din cele 6 pipeline-uri)
- Coerență cache MESIF (https://en.wikipedia.org/wiki/MESIF_protocol)
- Ierarhia chache-ului face ca procesoare diferite să aibă costuri diferite de acces la zone diferite de memorie => ***NUMA***!

Sisteme paralele cu memorie distribuită



Distribuția memoriei

- Avantaje
  - Memoria scalează cu numărul de procesoare. Creșterea numărului de procesoare este direct proporțională cu creșterea dimenisunii memoriei.
  - Fiecare procesor accesează rapid memoria proprie fără să interfereze și fără costuri suplimentare necesare pentru menținerea coerenței cache-ului.
  - Cost mic: putem folosi procesoare comune + rețea
- Dezavantaje
  - Programatorul este responsabil de modul în care se realizează comunicația între procesoare.
  - Poate fi dificil de alocat structuri de date existente într-o memorie globala pe o organizare distribuită.
  - Timpi NUMA (Non-uniform Memory Access)

Comparație între arhitecturile cu memorie partajată și distribută

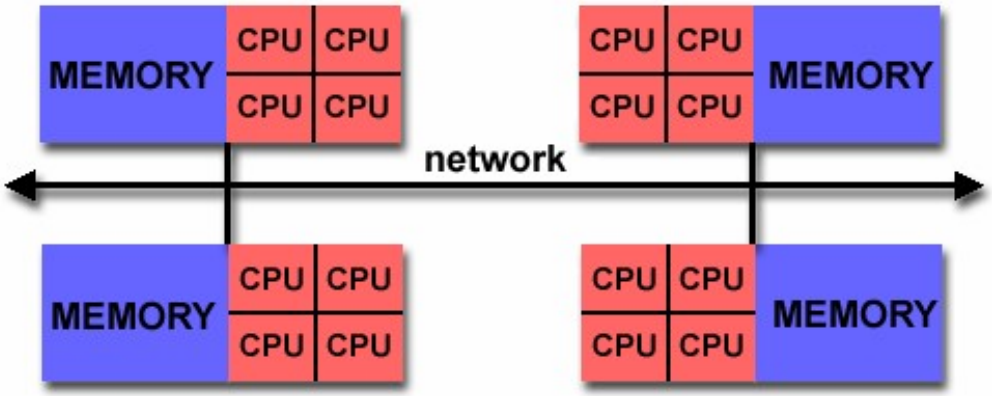
Arhitectură	CC-UMA	CC-NUMA	Distribuită
Exemple	SMP Sun Vexx DEC/Compaq SGI Challenge IBM POWER3	Bull NovaScale SGI Origin Sequent HP Exemplar DEC/Compaq IBM POWER4 (MCM)	Cray T3E Maspar IBM SP2 IBM BlueGene
Comunicație	MPI Threads OpenMP shmem	MPI Threads OpenMP Shmem	MPI
Scalabilitate	x10 de procesoare	X100 de procesoare	X1000 de procesoare
Limitări	Lățimea de bandă Memorie-CPU	Lățimea de bandă Memorie-CPU șu NUMA	Admimistraea și programarea sunt dificil de dezvoltat și întreținut
Disponibilitate SW	Mii de ISVs	Mii de ISVs (*Indep. Sw. Vendor*)	Sute de ISVs

IBM BlueGene (https://en.wikipedia.org/wiki/IBM_Blue_Gene)

- 3 generații care au condus în https://www.top500.org/ :
  - BG/L: Feb 1999
  - BG/P: June 2007
  - BG/Q: Nov 2011
  - 2015 s-a oprit dezvoltarea
- 15 instalări
- Aplicații
  - Jocul de șah
  - Simularea a 1% din cortex (1.6 bn de neuroni cu approx. 9 tn de conexiuni)

Arhitecturi hibride de memorie

- Cele mai mari și mai rapide sisteme din lume folosesc atât memoria partajată cât si cea distribuită.
- Componentele cu memorie partajată sunt mașini CC-SMP (**C**ache **C**oherent **S**ymetric **M**ultiprocessor); procesoarele accesează memoria componentei în mod global.



- Memoria distribuită este obținută prin conectarea în rețea a mai multor SMP-uri. Deoarece fiecare SMP are acces direct numai la memoria sa (NU la memoria altui SMP), este nevoie de comunicație de date pentru a transfera date de la un SMP la altul.
- Trend-ul actual arată că aceste sisteme sunt cele care vor domina domeniul HPC.
- Avantajele si dezavantajele: aspectele comune pentru arhitecturile cu memorie parajată și distribuită.

Multicalculatoare (Message-Passing MIMDs)

- Procesoarele sunt conectate în rețea
  - Poate fi conexiune Etherner sau o magistrală.
- Fiecare procesor are memorie locală si poate accesa doar memoria locală.
- Datele sunt transmise între procesoare prin mesaje, asa cum este specificat în aplicație printr-un limbaj adecvat (e.g., MPI, PVM).
- Problema este împărțită in task-uri care se execută concurent pe procesoare.
- Unui procesor îi pot fi alocate mai multe task-uri.

Dezavantaje ale multicalculatoarelor

Sunt derivate din dezavantajele transmiterii de mesaje:

- Programatorul trebuie să apeleze explicit codul care transmite mesajul.
- Programarea de nivel jos (i.e., explicită) predispune la erori.
- Datele nu sunt partajate ci sunt copiate, ceea ce duce la creșterea dimeniunii totale a datelor administrate.
- Integritatea datelor: este dificil de menținut corectitudinea datelor atunci când administrăm mai multe copii ale aceleiași unități de memorie.

Avantaje ale multicalculatoarelor

- Ușurința accesului la date.
- Permite ca sisteme diferite să proceseze aceleași date în mod independent.
- Permite updrade ușor al sistemelor atunci când avem la dispoziție procesoare mai rapide.
- Arhitecturile hibride (cluster de SMP-uri) sunt o soluție la îndemână
  - Asigură în același timp partajarea și distribuția.

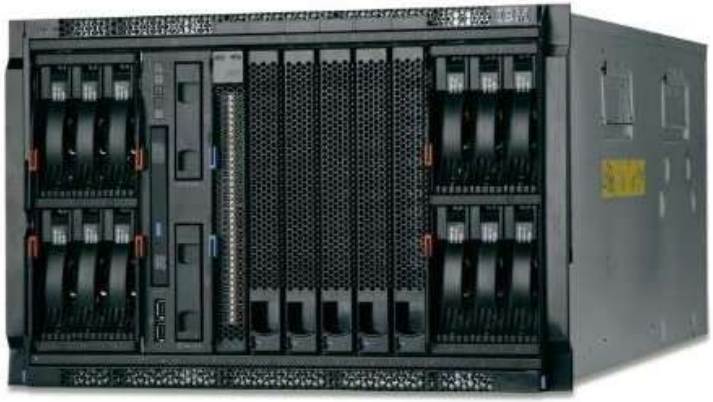
Rolul arhitecturii de calcul paralel/distribuit

- Proiectarea și implementarea unei arhitecturii de calcul paralel/distribuit este să maximizeze performanța și programabilitatea (i.e., usurința de dezvoltare/testare/depanare/intreținere) sistemului în limitele impuse de tehnologie și cost.
- Paralelismul arhitectural
  - Se poate aplica la toate nivelel unui sistem.
  - Oferă o persepctivă deosebită asupra arhitecturii unui sistem.
  - Tradițional, elementele sistemului (procesoare și memorii) sunt colocalizate.
  - Creșterea lărgimii de bandă permite paralelism la distanțe mai mari.

Tendințe tehnologice

- Tehnologiile actuale continuă să crească densitatea chip-urilor.
- Frecvența de lucru pentru arhitecturile single-core necesită prea multă energie.
- Unitilizarea tehnologiilor/sistemelor obișnuite (en., *comodity*) sau gata de utilizat (en., *off-the-shelf*) necesită costuri scăzute.
- Microprocesoarele multi-core (i.e., AMD, IBM, Intel) sunt comune.
- Lățimea de bandă mare (i.e., 10GB Ethernet) este în general disponibilă.

Tendințe arhitecturale



IBM BladeCenter S Chassis

- Împachetarea soluțiilor paralele

în *șasiuri* comune

  - Servere Blade (i.e., IBM, Dell, HP, etc.)

- SW
  - Windows Compute Cluster Server 2003 apărut în 2006 este prima tehnologie HPC oferită de Microsoft.
  - In prezent avem Windows Server 2019 care permite sistemelor server să lucreze împreună ca un cluster.

D.p.d.v. economic

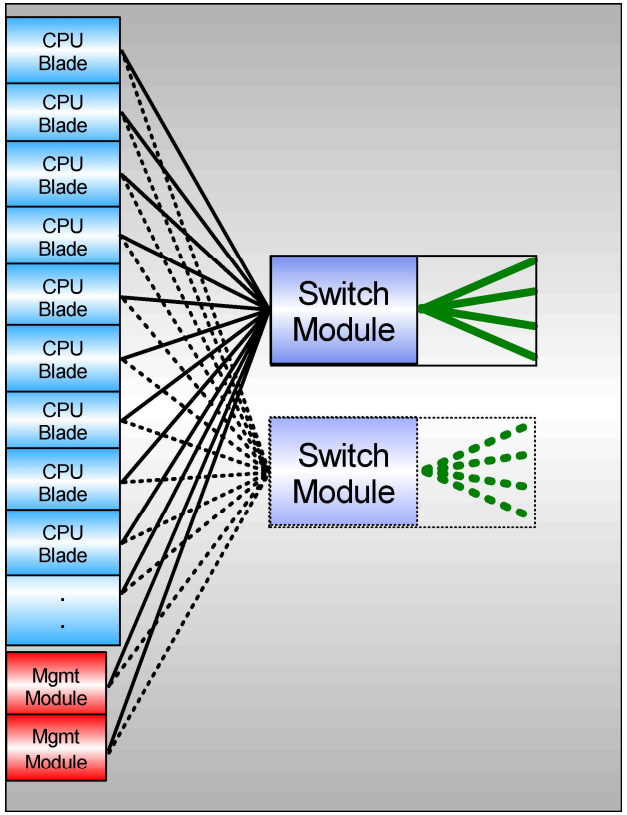
- Reducerea costurilor serverelor cu bandă largă de interconectare determină din ce în ce mai multe aplicații să devină paralelizabile.
- Microprocesoarele comune sunt rapide și ieftine
  - Dezvoltarea lor costă zeci de milioane de USD
  - DAR, se vand în cantități foarte mari, mult mai mari decât supercalculatoarele.
- Standardizarea face ca SMP-urile să devină un lucru comun
- Sistemele desktop
  - Mai multe procesoare mai mici, sau unul mai mare?
  - Arhitectură multiprocesor pe un singur chip.

Exemplu: Blade server

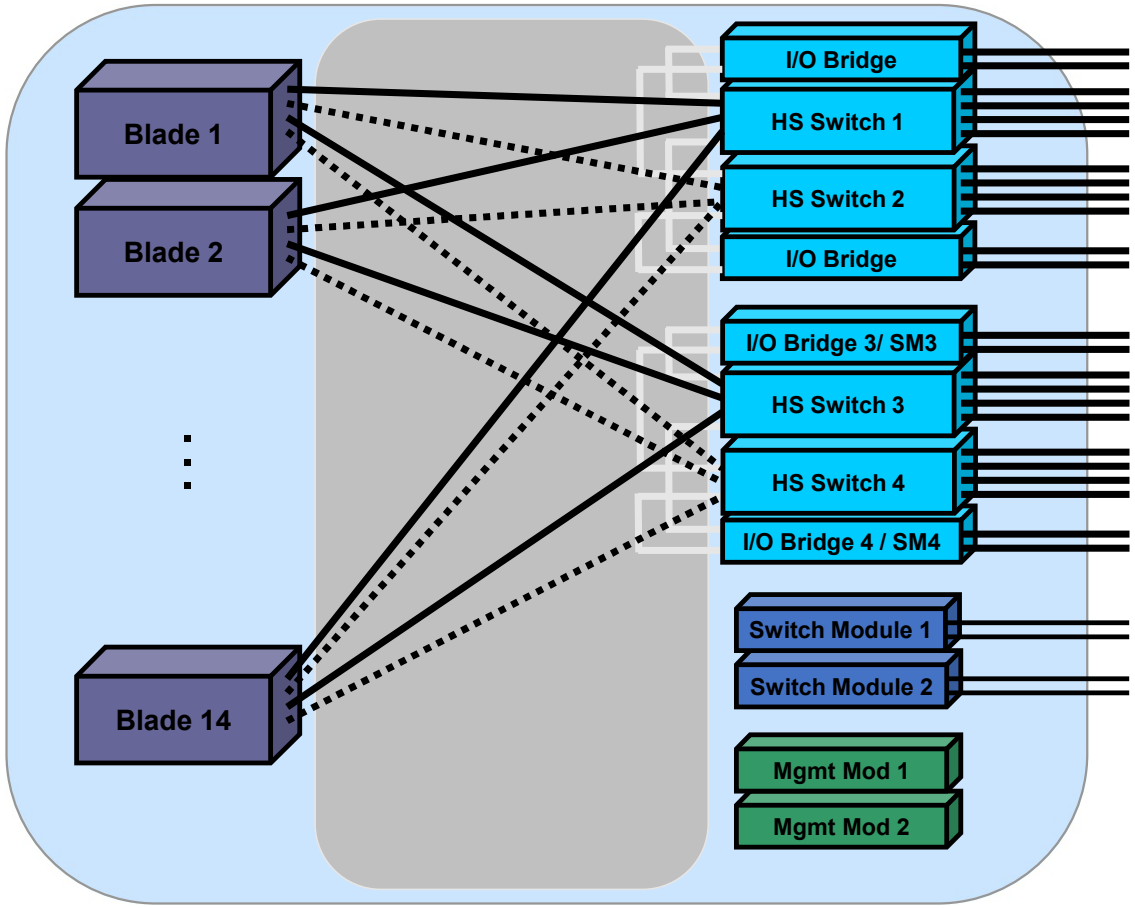
- 14 Blades intr-un 9U Chassis Form Factor
  - IBM BladeCenter HS22 7870 2x QC Xeon X5560 2.8GHz 24GB costa approx. 200 euro.
- Rețea de înaltă performanță
- 4 surse de 2900W
- Descriere
  - https://lenovopress.com/tips0996-bladecenter-h

IBM BladeCenter H Chassis

Prezentare generală BladeCenter



Prezentare generală BladeCenter H



- Switch de mare viteză
  - Ethernet
  - 4x (16 wire) blade links
  - 4x (16 wire) bridge links
  - 1x (4 wire) Mgmt links
  - Uplinks: Up to 12x links for IB and at least four 10Gb links for Ethernet
- I/O Bridge
  - Ethernet, Fible Channel
  - Dual 4x (16 wire) cablare internă către fiecare HSSM

Arhitectura sistemelor distribuite

Tipuri:

- Arhitecturi de tip client-server
- Arhitecturi cu obiecte distribuite
  - CORBA: ORB (Object request brokers)
- Arhitecturi *peer-to-peer* și *service-oriented*

Distribuția = Procesarea are loc pe mai multe sisteme, în nici un caz pe o singură mașină.

Ingineria aplicațiilor distribuite este pentru *enterprise computing systems*.

Arhitectura sistemelor distribuite (cont.)

- Arhitecturile client-server
  - Serviciile sunt distribuite și sunt denumite client.
  - Serverele care oferă servicii au tratament diferit față de clienții care utilizează serviciile.
- Arhitecturile cu obiecte distribuite
  - NU există nici o distincție între clienți si servere
  - Orice obiect din sistem poate oferi sau folosi servicii ale altor obiecte.
- *Middleware*
  - Software care administrează și susține diferite componente ale sistemului distribuit (i.e., se află în *mijlocul* sistemului).
  - Este de obicei SW de tip *off-the-shelf* (i.e., CORBA, Java RMI, SOAP, EJB), adică nu este special scris.

Arhitectura sistemelor distribuite (cont.)

- Arhitecturile multiprocesor
  - Cele mai simple sisteme distribuite.
  - Distribuția proceselor pe procesoare este sub controlul unui dispecer (en., *dispatcher)*.
- Arhitecturile client-server
  - Aplicația = mulțime de servicii oferite de servere + mulțime de clienți care folosesc/apelează serviciile.
  - Clienții știu/cunosc serverele, dar serverele nu trebuie să aibă cunoștință de clienți.

Arhitectura sistemelor distribuite (cont.)

- Arhitecturile client-server (cont.)
  - Clienții și serverele sunt procese logice (pot avea SO diferite!).
    - Diferențele de nivel jos (SO, limbaj de programare) sunt irelevante dacă serverele și clienții folosesc aceleași protocoale de comunicație și au aceleași aplicații.
  - Alocarea proceselor la procesoare nu este neapărat 1:1.
  - SW de comunicație este cel care permite clienților să interacționeze cu serverele: TCP/IP.
  - Funcționalitățile aplicației pot fi împărțite între cleint și server pentru a optimiza utilizarea resurselor.
  - Clase de aplicații: Host-based, Server based, Client-based, Cooperativ.

Arhitectura sistemelor distribuite (cont.)

- Arhitecturile client-server (cont)
  - Clase de aplicații
    - Host-based: Toată procesarea este efectuată pe server. Clientul este doar un terminal.
    - Server-based: Serverul efectueaza procesarea, dar clientul are GUI pentru interacțiune.
    - Cooperative: Atât serverul cât și clentul rulează logică din aplicație.
      - Complex de setat si întreținut, dar oferă productivitate și eficiență mare.
    - Client-based: Toată procesarea (View+Controller+Model logic) se efectuază la client.
      - Serverul realizează doar validarea datelor și funcțiile pe baza de date.
      - Este cea mai utilizată arhitectură pentru aplicațiile desktop.
      - Nivelele devin: Client, server de aplicații (middle-tier server), Back-end server.

Arhitectura sistemelor distribuite (cont.)

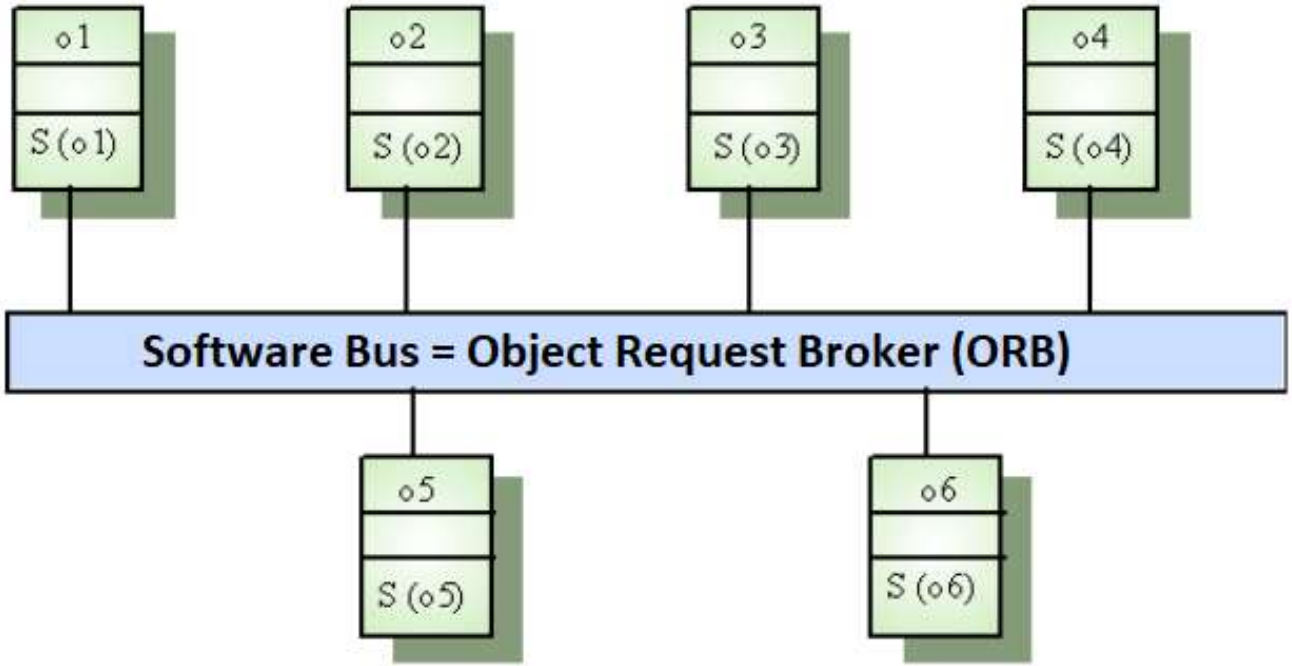
- Arhitecturile client-server (cont.)
  - Sunt de obicei pe 3 nivele
    - View/prezentare – client [*thin/fat*]
    - Controller/application processing – server
    - Model/administrarea datelor – server
    - Exemplu: Homebank = Client [mobile/web] – Web Server – Database server
  - Tipuri de servere
    - Servere de baze de date
    - Servere de aplicații
    - Servere de resurse
    - Servere de calcul (en., computation)

Arhitectura sistemelor distribuite (cont.)

- Arhitecturile client-server (cont)
  - Avantaje:
    - Acces facil la baze de date
    - Partajarea resurselor HW si SW
    - Scalabilitatea
    - Rapiditatea în dezvoltare
  - Limitări
    - Costuri de implementare: echipamente, integrare, timp mare
    - Tehnologii complexe
    - Unelte de administrare necustomizabile
    - Securiate
    - Roluri multiple: admin, owner, proiectant, utilizator, etc.

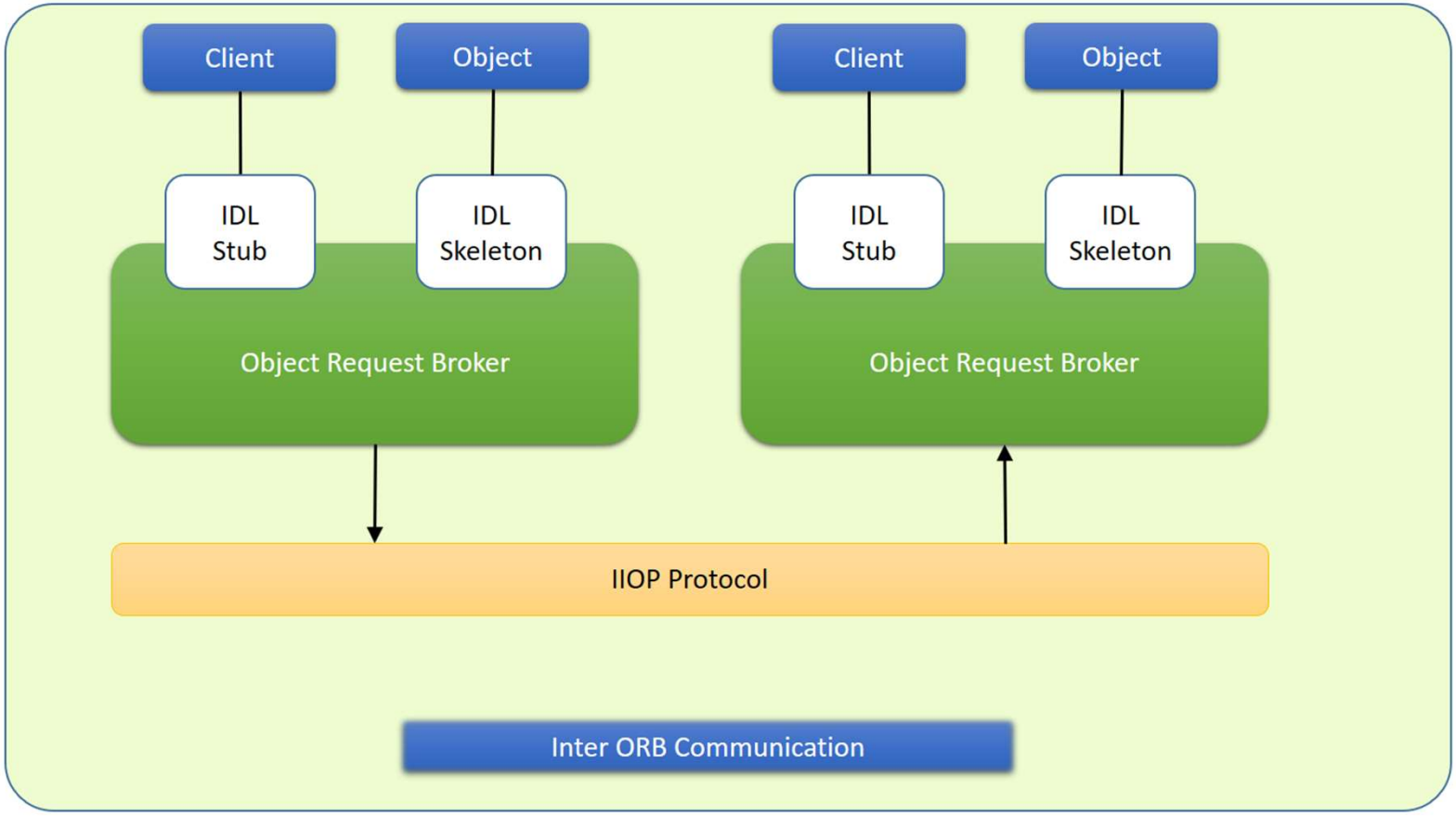
Arhitectura sistemelor distribuite (cont.)

- Arhitecturile cu obiecte distribuite

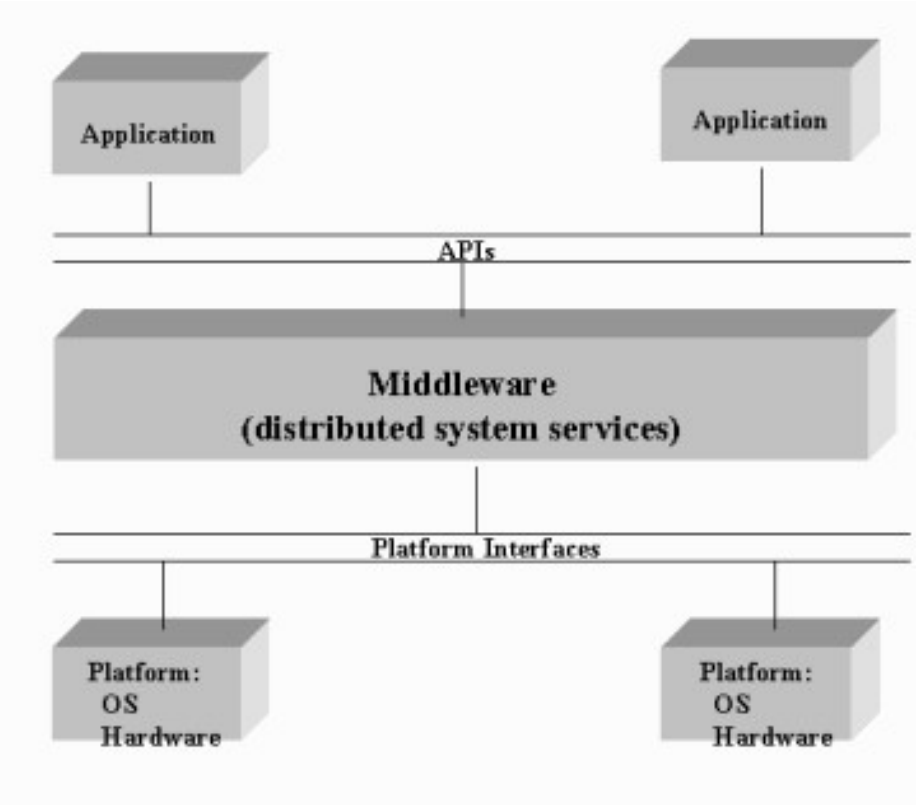


Arhitectura sistemelor distribuite (cont.)

- Arhitecturile cu obiecte distribuite (cont.)



Arhirectura SD: Middleware d.p.d.v. logic



Fiabiltatea sistemelor distribuite

- Transmiterea de mesaje fiabilă garantează livrarea datelor (mesaje, obiecte, fișiere, etc.)
  - Uneori nu este nevoie să confirmăm clientului că cererea/datele/etc. au ajuns la server, dar este util.
- Opțiunea 1. Dacă transmirerea eșuează atunci procesul care a trimis cererea este notificat.
- Opțiunea 2. Mesajele/datele sunt trimise fără a primi înapoi informații despre succes sau eșec (i.e., TCP vs. UDP)
  - Acească opțiune reduce complexitatea și costurile suplomentare de comunicație.
- Aplicațiile care necesită confirmarea ca mesajul a fost transmis, confirmarea însăși va fi un mesaj.

Fialbiltatea sistemelor distribuite (cont.)

Cu blocare vs. Fără blocare

- Fără blocare (asincronă)
  - După ce un proces a trimis un Send/Receive acesta nu se blochează
  - Mod eficient și flexibil de a utiliza transmiterea mesajelor de către procese
  - Testarea și depanarea este dificilă deoarece este dificil de reprodus contextul (i.e., secvența/ordinea exactă de desfășutrare în timp a operațiilor)
- Cu blocare
  - Reprezintă varianta sincronă de comunicare
  - Procesul care trimite cererea nu cedează controlul până când mesajul nu a fost transmis (i.e., trimis si primită confirmarea de recepționare)
  - Procesul care primește mesajul nu cedează controlul (i.e., nu continuă) până când acesta nu deține efectiv mesajul sau mesajul se află într-un buffer.

Apelurile la distanță (en., *remote procedure call*)

- Permit programelor de pe sisteme diferite să interacționeze prin apelări simple.
- Sunt folosite pentru a accesa servicii aflate la distanță
- Larg acceptate și des folosite pentru a încapsula comunicația în sistemele distribuite.
- Codul de comunicare al unei aplicații se generează automat.
- Modulele client și server pot fi mutate de pe o platformă (i.e., mașină + SO) pe alta cu modificări minime.

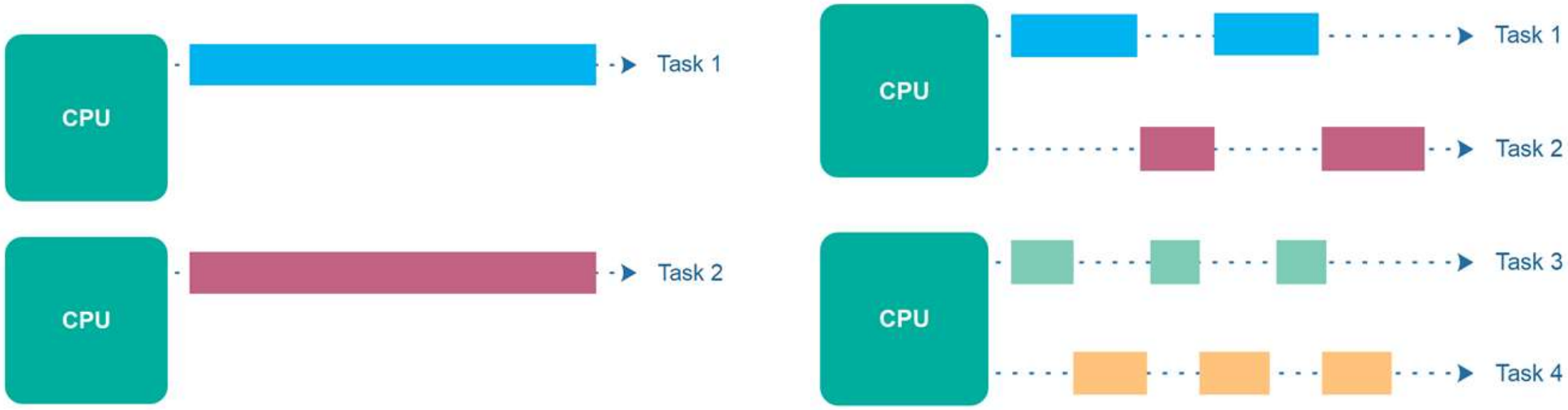
Paralelismul in Java [*bazat pe Task-uri*]

- Este asemănător cu cel din C++.
  - Și multithreading-ul este asemănător cu cel din C/C++.
- Aceeași abordare:
  - Crearea unui Task
  - Terminarea unui Task
  - Realizarea grafului de calcul (en., *computation graph*) folosind Fork/Join
- Task-urile rulate în paralel sunt independente (i.e., ordinea de execuție nu contează).
- Task-urile sunt definite în funcție de logica pe care o execută (paralelism funcțional) sau de datele utilizate în cadrul procesării (paralelism al domeniului).

Paralelismul in Java [bazat pe Task-uri] (cont.)

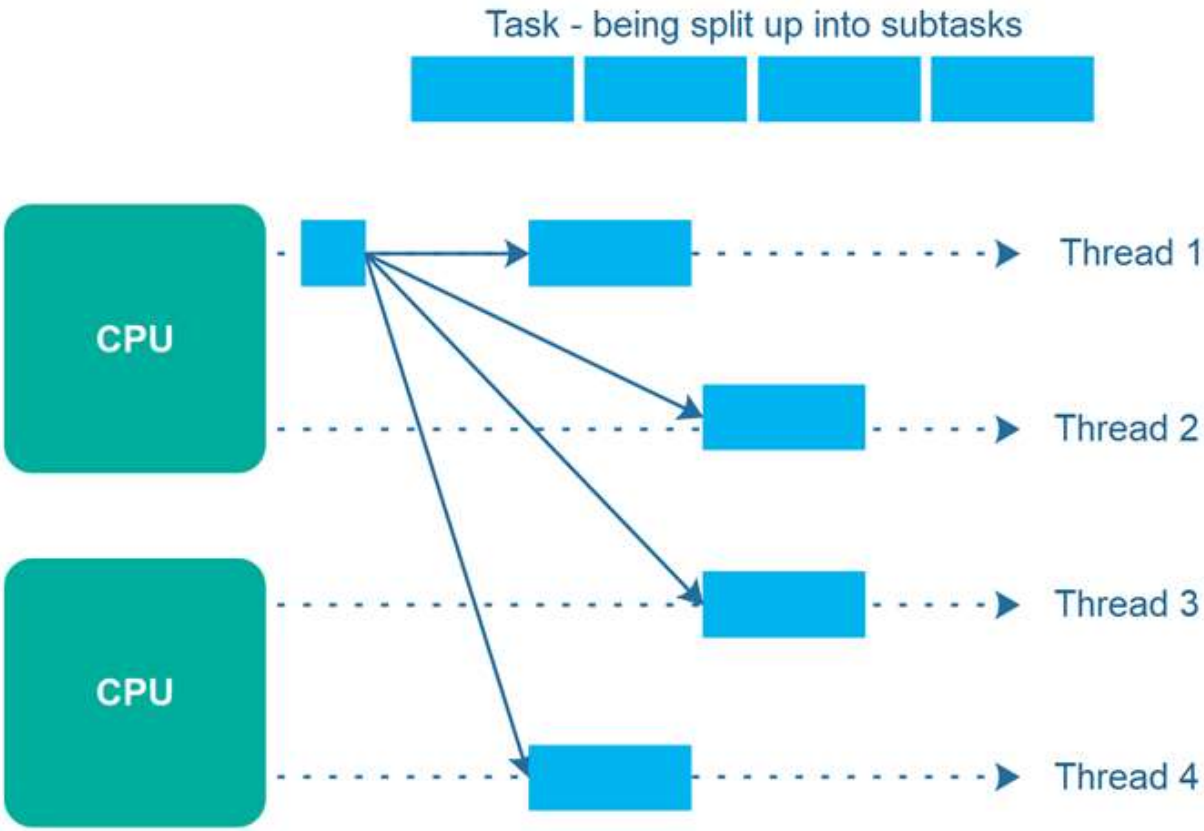
- Java SE pune la dispoziție *Fork/Join Framework* pentru a putea implementa paralelismul cu ușurință în aplicații.
- Dificultăți
  - Utilizarea colecțiilor care nu sunt *thread-safe*, ceea ce înseamnă că mai multe fire nu pot accesa o colecție fără a produce interferențe sau erori de consistență a memoriei.
  - *Obs*: Paralelismul nu este automat mai rapid decât varianta secvențială, dar poate fi dacă aveți suficiente date și procesoare.
  - Fluxurile (en., *streams*) în Java pot rula serial sau paralel. În situația rulării paralele, Java RTE (runtime) prin JVM partiționează fluxul în mai multe sub-fluxuri.

Execuția paralelă Vs. Execuția paralelă concurentă



Paralelism

- Se referă la o aplicație care împarte task-urile în sub-task-uri mai mici care se execută în paralel pe mai multe procesoare în același timp.
  - NU se referă la execuția paralelă concurentă.



Paralelismul task-urilor

- Task-uri diferite pot folosi sau nu aceleași date
  - De aici o accelerare mai mică
- Procesarea este asincronă
- Gradul de paralelizare este proporțional cu numărul de task-uri independente obținut/creat.
- Echilibrarea încărcării depinde de diponibilitatea HW li de algoritmii de planificare (statici sau dinamici)

Fork/Join Framework în Java

- Pachetul *java.util.concurrent* include clase și interfețe care susțin programarea paralelă.
  - Simplifică procesul de creare a firelor, utilizare a lor, automatizează mecanismul de alocare a proceselor pe procesoare.
- Clase de bază

ForkJoinTask<V>: este o clasă abstractă care definește un task.

- Metoda fork():
  - crează un task (în mod asemenător cu cu un fir creat de clasa Thread).
  - Permite execuția asincronă a task-ului.
- Metoda join():
  - Permite așteptarea până când task-ul apelant se termină.
- Metoda invoke():
  - Combină fork și join într-un singur apel.

Fork/Join Framework în Java (cont.)

- ForkJoinPool: este o clasă care permite administrarea și monitorizarea execuției de taskuri ForkJoinTask.
- RecursiveAction: este o derivare abstractă a clasei ForkJoinTask. De obicei, derivăm această clasă pentru a crea un task care NU returnează un rezultat sau returnează *void*. Metoda compute() definită în această clasă este suprascrisă pentru a conține procesarea/codul task-ului.
- RecursiveTask<V>: este o derivare abstractă a clasei ForkJoinTask. De obicei, derivăm această clasă pentru a crea un task care returnează un rezultat. Metoda compute() definită în această clasă este suprascrisă pentru a conține procesarea/codul task-ului.

Strategia Fork/Join Framework

- Framework-ul folosește strategia *divide and conquer* recursivă pentru a implementa procesarea paralelă.
  - Împarte un task în subtask-uri mai mici; apoi, fiecare subtask în alte subtast-uri mai mici, etc.
  - Împărțirea este aplicată recursiv până când fiecare subtask este suficient de mic pentru a fi executat secvențial.
- Exemplu: Incrementarea valorilor dintr-un tablou de N numere.
  - Strategia *divide and conquer* poate fi aplicată până când obținem subtask-uri de dimensiune 1 (i.e., un element din tablou). Apoi, subproblemele de dimensiune 1 sunt executate în paralel de procesoarele disponibile.
  - Pe un sistem secvențial (cu un procesor) ar trebui să parcurgem întreg tabloul pentru efectuarea procesării (i.e., incrementării), ceea ce at fi ineficient d.p,d,v, al procesării paralele.

Strategia Fork/Join Framework (cont.)

Operația de invocare/apleare în paralel se realizează prin:

- fork: crează două sau mai multe task-uri.
- join: întrerupe/suspendă firul curent (i.e., apelant) până când task-ul nou se finalizează.

Programarea concurentă în Java

- Cuvântul cheie synchronized în Java asigură:
  - că doar un singur fir poate executa un bloc de cod în același timp
  - că fiecare fir care intră într-un bloc de cod sincronizat vede efectele tuturor modificărilor anterioare (i.e., ultima variantă) care erau păzite de aceeași blocare
- Sincronizarea este necesară pentru accesul reciproc exclusiv la blocuri și pentru o comunicare fiabilă între fire.
- Utilizarea cuvântul cheie synchronized pentru definirea unei metode
  - asigură că un singur fir poate intra în această metodă la un moment dat. Un alt fir care apelează această metodă trebuie să aștepte până când primul fir părăsește metoda.

Programarea concurentă în Java

Clase de bază

- java.lang.Thread: implementează interfața java.lang.Runnable care definește metoda run().
  - Metoda run() este apelată de obiectul de tip Thread care conține codul care trebuie executat.
  - Runnable reprezintă sarcina care trebuie rulată iar Thread este task-ul care rulează sarcina.
- java.util.concurrent.ExecutorService: interfață care este un înlocuitor de nivel superior pentru lucrul direct cu fire.
  - Pot rula sarcini asincrone și pot admimistra un grup/pool de fire care nu trebuie create manual.

Programarea concurentă în Java (cont.)

- java.util.concurrent.Executor: interfață care permite obiectelor care o implementează sa execute sarcini Runnable.
  - Se oferă astfel o modalitatea de a decupla/separa trimiterea task-ului de mecanica executării task-ului (i.e., utilizarea firului, planificarea, etc.)
  - Exemplu:

  În loc de apelul Thread(new(RunnableTask())).start() necesar fiecărui task

Putem avea secvența:	cu un executor care rulează task-ul imediat

Executor executor = anExecutor;
executor.execute(new RunnableTask1());
executor.execute(new RunnableTask2());
...
	class DirectExecutor implements
	Executor {
	public void execute(Runnable r) {
	r.run();
	}
	}

Programarea concurentă în Java (cont.)

- Totuși, de obicei task-urile sunt executate de un fir separat, nu de firul apelant. Acest executor crează cate un fir nou pentru fiecare task.

```
class ThreadPerTaskExecutor implements Executor {
    public void execute(Runnable r) {
        new Thread(r).start();
    }
}
```

Fluxuri Java în paralel

- Un flux (i.e., stream) poate fi executat serial sau paralel.
- Atunci cand dorim executie paralela Java RTE (runtime) partiționează fluxul in mai multe sub-fluxuri.
- Astfel, subfluxurile se execută în paralel iar rezultatele sunt agregate.

default Stream<E> parallelStream()	S parallel()
Returns a possibly parallel Stream with this collection as its source.	*Returns an equivalent stream that is parallel. May return itself, either because the stream was already parallel, or because the underlying stream state was modified to be parallel.* *This is an intermediate operation.*